C H A P T E R 3

Relational Model

The relational model is today the primary data model for commercial data-processing applications. It has attained its primary position because of its simplicity, which eases the job of the programmer, as compared to earlier data models such as the network model or the hierarchical model.

In this chapter, we first study the fundamentals of the relational model, which provides a very simple yet powerful way of representing data. We then describe three formal query languages; query languages are used to specify requests for information. The three we cover in this chapter are not user-friendly, but instead serve as the formal basis for user-friendly query languages that we study later. We cover the first query language, relational algebra, in great detail. The relational algebra forms the basis of the widely used SQL query language. We then provide overviews of the other two formal languages, the tuple relational calculus and the domain relational calculus, which are declarative query languages based on mathematical logic. The domain relational calculus is the basis of the QBE query language.

A substantial theory exists for relational databases. We study the part of this theory dealing with queries in this chapter. In Chapter 7 we shall examine aspects of relational database theory that help in the design of relational database schemas, while in Chapters 13 and 14 we discuss aspects of the theory dealing with efficient processing of queries.

3.1 Structure of Relational Databases

A relational database consists of a collection of **tables**, each of which is assigned a unique name. Each table has a structure similar to that presented in Chapter 2, where we represented E-R databases by tables. A row in a table represents a *relationship* among a set of values. Since a table is a collection of such relationships, there is a close correspondence between the concept of *table* and the mathematical concept of

relation, from which the relational data model takes its name. In what follows, we introduce the concept of relation.

In this chapter, we shall be using a number of different relations to illustrate the various concepts underlying the relational data model. These relations represent part of a banking enterprise. They differ slightly from the tables that were used in Chapter 2, so that we can simplify our presentation. We shall discuss criteria for the appropriateness of relational structures in great detail in Chapter 7.

3.1.1 Basic Structure

Consider the *account* table of Figure 3.1. It has three column headers: *account-number*, *branch-name*, and *balance*. Following the terminology of the relational model, we refer to these headers as **attributes** (as we did for the E-R model in Chapter 2). For each attribute, there is a set of permitted values, called the **domain** of that attribute. For the attribute *branch-name*, for example, the domain is the set of all branch names. Let D_1 denote the set of all account numbers, D_2 the set of all branch names, and D_3 the set of all balances. As we saw in Chapter 2, any row of *account* must consist of a 3-tuple (v_1, v_2, v_3), where v_1 is an account number (that is, v_1 is in domain D_1), v_2 is a branch name (that is, v_2 is in domain D_2), and v_3 is a balance (that is, v_3 is in domain D_3). In general, *account* will contain only a subset of the set of all possible rows. Therefore, *account* is a subset of

$$D_1 \times D_2 \times D_3$$

In general, a **table** of n attributes must be a subset of

$$D_1 \times D_2 \times \cdots \times D_{n-1} \times D_n$$

Mathematicians define a **relation** to be a subset of a Cartesian product of a list of domains. This definition corresponds almost exactly with our definition of *table*. The only difference is that we have assigned names to attributes, whereas mathematicians rely on numeric "names," using the integer 1 to denote the attribute whose domain appears first in the list of domains, 2 for the attribute whose domain appears second, and so on. Because tables are essentially relations, we shall use the mathematical

account-number	*branch-name*	*balance*
A-101	Downtown	500
A-102	Perryridge	400
A-201	Brighton	900
A-215	Mianus	700
A-217	Brighton	750
A-222	Redwood	700
A-305	Round Hill	350

Figure 3.1 The *account* relation.

account-number	*branch-name*	*balance*
A-101	Downtown	500
A-215	Mianus	700
A-102	Perryridge	400
A-305	Round Hill	350
A-201	Brighton	900
A-222	Redwood	700
A-217	Brighton	750

Figure 3.2 The *account* relation with unordered tuples.

terms **relation** and **tuple** in place of the terms **table** and **row**. A **tuple variable** is a variable that stands for a tuple; in other words, a tuple variable is a variable whose domain is the set of all tuples.

In the *account* relation of Figure 3.1, there are seven tuples. Let the tuple variable t refer to the first tuple of the relation. We use the notation t[*account-number*] to denote the value of t on the *account-number* attribute. Thus, t[*account-number*] = "A-101," and t[*branch-name*] = "Downtown". Alternatively, we may write t[1] to denote the value of tuple t on the first attribute (*account-number*), t[2] to denote *branch-name*, and so on. Since a relation is a set of tuples, we use the mathematical notation of $t \in r$ to denote that tuple t is in relation r.

The order in which tuples appear in a relation is irrelevant, since a relation is a *set* of tuples. Thus, whether the tuples of a relation are listed in sorted order, as in Figure 3.1, or are unsorted, as in Figure 3.2, does not matter; the relations in the two figures above are the same, since both contain the same set of tuples.

We require that, for all relations r, the domains of all attributes of r be atomic. A domain is **atomic** if elements of the domain are considered to be indivisible units. For example, the set of integers is an atomic domain, but the set of all sets of integers is a nonatomic domain. The distinction is that we do not normally consider integers to have subparts, but we consider sets of integers to have subparts—namely, the integers composing the set. The important issue is not what the domain itself is, but rather how we use domain elements in our database. The domain of all integers would be nonatomic if we considered each integer to be an ordered list of digits. In all our examples, we shall assume atomic domains. In Chapter 9, we shall discuss extensions to the relational data model to permit nonatomic domains.

It is possible for several attributes to have the same domain. For example, suppose that we have a relation *customer* that has the three attributes *customer-name*, *customer-street*, and *customer-city*, and a relation *employee* that includes the attribute *employee-name*. It is possible that the attributes *customer-name* and *employee-name* will have the same domain: the set of all person names, which at the physical level is the set of all character strings. The domains of *balance* and *branch-name*, on the other hand, certainly ought to be distinct. It is perhaps less clear whether *customer-name* and *branch-name* should have the same domain. At the physical level, both customer names and branch names are character strings. However, at the logical level, we may want *customer-name* and *branch-name* to have distinct domains.

One domain value that is a member of any possible domain is the **null** value, which signifies that the value is unknown or does not exist. For example, suppose that we include the attribute *telephone-number* in the *customer* relation. It may be that a customer does not have a telephone number, or that the telephone number is unlisted. We would then have to resort to null values to signify that the value is unknown or does not exist. We shall see later that null values cause a number of difficulties when we access or update the database, and thus should be eliminated if at all possible. We shall assume null values are absent initially, and in Section 3.3.4, we describe the effect of nulls on different operations.

3.1.2 Database Schema

When we talk about a database, we must differentiate between the **database schema**, which is the logical design of the database, and a **database instance**, which is a snapshot of the data in the database at a given instant in time.

The concept of a relation corresponds to the programming-language notion of a variable. The concept of a **relation schema** corresponds to the programming-language notion of type definition.

It is convenient to give a name to a relation schema, just as we give names to type definitions in programming languages. We adopt the convention of using lowercase names for relations, and names beginning with an uppercase letter for relation schemas. Following this notation, we use *Account-schema* to denote the relation schema for relation *account*. Thus,

$$Account\text{-}schema = (account\text{-}number, branch\text{-}name, balance)$$

We denote the fact that *account* is a relation on *Account-schema* by

$$account(Account\text{-}schema)$$

In general, a relation schema consists of a list of attributes and their corresponding domains. We shall not be concerned about the precise definition of the domain of each attribute until we discuss the SQL language in Chapter 4.

The concept of a **relation instance** corresponds to the programming language notion of a value of a variable. The value of a given variable may change with time; similarly the contents of a relation instance may change with time as the relation is updated. However, we often simply say "relation" when we actually mean "relation instance."

As an example of a relation instance, consider the *branch* relation of Figure 3.3. The schema for that relation is

$$Branch\text{-}schema = (branch\text{-}name, branch\text{-}city, assets)$$

Note that the attribute *branch-name* appears in both *Branch-schema* and *Account-schema*. This duplication is not a coincidence. Rather, using common attributes in relation schemas is one way of relating tuples of distinct relations. For example, suppose we wish to find the information about all of the accounts maintained in branches

branch-name	*branch-city*	*assets*
Brighton	Brooklyn	7100000
Downtown	Brooklyn	9000000
Mianus	Horseneck	400000
North Town	Rye	3700000
Perryridge	Horseneck	1700000
Pownal	Bennington	300000
Redwood	Palo Alto	2100000
Round Hill	Horseneck	8000000

Figure 3.3 The *branch* relation.

located in Brooklyn. We look first at the *branch* relation to find the names of all the branches located in Brooklyn. Then, for each such branch, we would look in the *account* relation to find the information about the accounts maintained at that branch. This is not surprising—recall that the primary key attributes of a strong entity set appear in the table created to represent the entity set, as well as in the tables created to represent relationships that the entity set participates in.

Let us continue our banking example. We need a relation to describe information about customers. The relation schema is

$$\mathit{Customer\text{-}schema} = (\mathit{customer\text{-}name}, \mathit{customer\text{-}street}, \mathit{customer\text{-}city})$$

Figure 3.4 shows a sample relation *customer* (*Customer-schema*). Note that we have omitted the *customer-id* attribute, which we used Chapter 2, because now we want to have smaller relation schemas in our running example of a bank database. We assume that the customer name uniquely identifies a customer—obviously this may not be true in the real world, but the assumption makes our examples much easier to read.

customer-name	*customer-street*	*customer-city*
Adams	Spring	Pittsfield
Brooks	Senator	Brooklyn
Curry	North	Rye
Glenn	Sand Hill	Woodside
Green	Walnut	Stamford
Hayes	Main	Harrison
Johnson	Alma	Palo Alto
Jones	Main	Harrison
Lindsay	Park	Pittsfield
Smith	North	Rye
Turner	Putnam	Stamford
Williams	Nassau	Princeton

Figure 3.4 The *customer* relation.

In a real-world database, the *customer-id* (which could be a *social-security* number, or an identifier generated by the bank) would serve to uniquely identify customers.

We also need a relation to describe the association between customers and accounts. The relation schema to describe this association is

$$Depositor\text{-}schema = (customer\text{-}name, account\text{-}number)$$

Figure 3.5 shows a sample relation *depositor* (*Depositor-schema*).

It would appear that, for our banking example, we could have just one relation schema, rather than several. That is, it may be easier for a user to think in terms of one relation schema, rather than in terms of several. Suppose that we used only one relation for our example, with schema

(*branch-name*, *branch-city*, *assets*, *customer-name*, *customer-street*
customer-city, *account-number*, *balance*)

Observe that, if a customer has several accounts, we must list her address once for each account. That is, we must repeat certain information several times. This repetition is wasteful and is avoided by the use of several relations, as in our example.

In addition, if a branch has no accounts (a newly created branch, say, that has no customers yet), we cannot construct a complete tuple on the preceding single relation, because no data concerning *customer* and *account* are available yet. To represent incomplete tuples, we must use *null* values that signify that the value is unknown or does not exist. Thus, in our example, the values for *customer-name*, *customer-street*, and so on must be null. By using several relations, we can represent the branch information for a bank with no customers without using null values. We simply use a tuple on *Branch-schema* to represent the information about the branch, and create tuples on the other schemas only when the appropriate information becomes available.

In Chapter 7, we shall study criteria to help us decide when one set of relation schemas is more appropriate than another, in terms of information repetition and the existence of null values. For now, we shall assume that the relation schemas are given.

We include two additional relations to describe data about loans maintained in the various branches in the bank:

customer-name	*account-number*
Hayes	A-102
Johnson	A-101
Johnson	A-201
Jones	A-217
Lindsay	A-222
Smith	A-215
Turner	A-305

Figure 3.5 The *depositor* relation.

loan-number	*branch-name*	*amount*
L-11	Round Hill	900
L-14	Downtown	1500
L-15	Perryridge	1500
L-16	Perryridge	1300
L-17	Downtown	1000
L-23	Redwood	2000
L-93	Mianus	500

Figure 3.6 The *loan* relation.

$$Loan\text{-}schema = (\textit{loan-number}, \textit{branch-name}, \textit{amount})$$
$$Borrower\text{-}schema = (\textit{customer-name}, \textit{loan-number})$$

Figures 3.6 and 3.7, respectively, show the sample relations *loan* (*Loan-schema*) and *borrower* (*Borrower-schema*).

The E-R diagram in Figure 3.8 depicts the banking enterprise that we have just described. The relation schemas correspond to the set of tables that we might generate by the method outlined in Section 2.9. Note that the tables for *account-branch* and *loan-branch* have been combined into the tables for *account* and *loan* respectively. Such combining is possible since the relationships are many to one from *account* and *loan*, respectively, to *branch*, and, further, the participation of *account* and *loan* in the corresponding relationships is total, as the double lines in the figure indicate. Finally, we note that the *customer* relation may contain information about customers who have neither an account nor a loan at the bank.

The banking enterprise described here will serve as our primary example in this chapter and in subsequent ones. On occasion, we shall need to introduce additional relation schemas to illustrate particular points.

3.1.3 Keys

The notions of *superkey*, *candidate key*, and *primary key*, as discussed in Chapter 2, are also applicable to the relational model. For example, in *Branch-schema*, {*branch-*

customer-name	*loan-number*
Adams	L-16
Curry	L-93
Hayes	L-15
Jackson	L-14
Jones	L-17
Smith	L-11
Smith	L-23
Williams	L-17

Figure 3.7 The *borrower* relation.

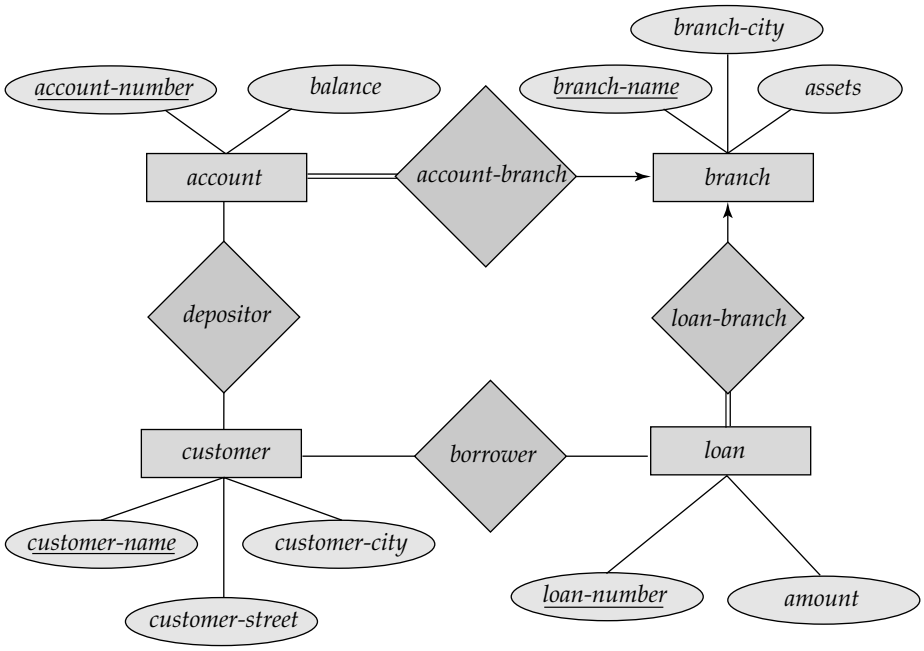


Figure 3.8 E-R diagram for the banking enterprise.

name} and {*branch-name, branch-city*} are both superkeys. {*branch-name*, *branch-city*} is not a candidate key, because {*branch-name*} is a subset of {*branch-name, branch-city*} and {*branch-name*} itself is a superkey. However, {*branch-name*} *is* a candidate key, and for our purpose also will serve as a primary key. The attribute *branch-city* is not a superkey, since two branches in the same city may have different names (and different asset figures).

Let *R* be a relation schema. If we say that a subset *K* of *R* is a *superkey* for *R*, we are restricting consideration to relations *r*(*R*) in which no two distinct tuples have the same values on all attributes in *K*. That is, if t_1 *and* t_2 are in *r* and $t_1 \neq t_2$, then $t_1[K] \neq t_2[K]$.

If a relational database schema is based on tables derived from an E-R schema, it is possible to determine the primary key for a relation schema from the primary keys of the entity or relationship sets from which the schema is derived:

- **Strong entity set**. The primary key of the entity set becomes the primary key of the relation.
- **Weak entity set**. The table, and thus the relation, corresponding to a weak entity set includes
  - The attributes of the weak entity set
  - The primary key of the strong entity set on which the weak entity set depends

The primary key of the relation consists of the union of the primary key of the strong entity set and the discriminator of the weak entity set.

- **Relationship set**. The union of the primary keys of the related entity sets becomes a superkey of the relation. If the relationship is many-to-many, this superkey is also the primary key. Section 2.4.2 describes how to determine the primary keys in other cases. Recall from Section 2.9.3 that no table is generated for relationship sets linking a weak entity set to the corresponding strong entity set.
- **Combined tables**. Recall from Section 2.9.3 that a binary many-to-one relationship set from A to B can be represented by a table consisting of the attributes of A and attributes (if any exist) of the relationship set. The primary key of the "many" entity set becomes the primary key of the relation (that is, if the relationship set is many to one from A to B, the primary key of A is the primary key of the relation). For one-to-one relationship sets, the relation is constructed like that for a many-to-one relationship set. However, we can choose either entity set's primary key as the primary key of the relation, since both are candidate keys.
- **Multivalued attributes**. Recall from Section 2.9.5 that a multivalued attribute M is represented by a table consisting of the primary key of the entity set or relationship set of which M is an attribute plus a column C holding an individual value of M. The primary key of the entity or relationship set, together with the attribute C, becomes the primary key for the relation.

From the preceding list, we see that a relation schema, say r_1, derived from an E-R schema may include among its attributes the primary key of another relation schema, say r_2. This attribute is called a **foreign key** from r_1, referencing r_2. The relation r_1 is also called the **referencing relation** of the foreign key dependency, and r_2 is called the **referenced relation** of the foreign key. For example, the attribute *branch-name* in *Account-schema* is a foreign key from *Account-schema* referencing *Branch-schema*, since *branch-name* is the primary key of *Branch-schema*. In any database instance, given any tuple, say t_a, from the *account* relation, there must be some tuple, say t_b, in the *branch* relation such that the value of the *branch-name* attribute of t_a is the same as the value of the primary key, *branch-name*, of t_b.

It is customary to list the primary key attributes of a relation schema before the other attributes; for example, the *branch-name* attribute of *Branch-schema* is listed first, since it is the primary key.

3.1.4 Schema Diagram

A database schema, along with primary key and foreign key dependencies, can be depicted pictorially by **schema diagram**s. Figure 3.9 shows the schema diagram for our banking enterprise. Each relation appears as a box, with the attributes listed inside it and the relation name above it. If there are primary key attributes, a horizontal line crosses the box, with the primary key attributes listed above the line. Foreign

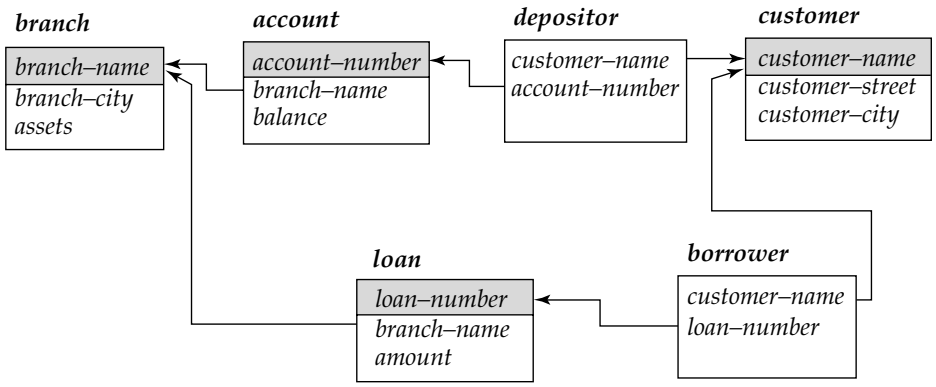


Figure 3.9 Schema diagram for the banking enterprise.

key dependencies appear as arrows from the foreign key attributes of the referencing relation to the primary key of the referenced relation.

Do not confuse a schema diagram with an E-R diagram. In particular, E-R diagrams do not show foreign key attributes explicitly, whereas schema diagrams show them explicity.

Many database systems provide design tools with a graphical user interface for creating schema diagrams.

3.1.5 Query Languages

A **query language** is a language in which a user requests information from the database. These languages are usually on a level higher than that of a standard programming language. Query languages can be categorized as either procedural or nonprocedural. In a **procedural language**, the user instructs the system to perform a sequence of operations on the database to compute the desired result. In a **nonprocedural language**, the user describes the desired information without giving a specific procedure for obtaining that information.

Most commercial relational-database systems offer a query language that includes elements of both the procedural and the nonprocedural approaches. We shall study the very widely used query language SQL in Chapter 4. Chapter 5 covers the query languages QBE and Datalog, the latter a query language that resembles the Prolog programming language.

In this chapter, we examine "pure" languages: The relational algebra is procedural, whereas the tuple relational calculus and domain relational calculus are nonprocedural. These query languages are terse and formal, lacking the "syntactic sugar" of commercial languages, but they illustrate the fundamental techniques for extracting data from the database.

Although we shall be concerned with only queries initially, a complete data-manipulation language includes not only a query language, but also a language for database modification. Such languages include commands to insert and delete tuples,